

JAVA INTERVIEW PREPARATION

CHAPTER 54

RESILIENT BATCH PROCESSING
WITH SPRING BATCH

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Resilient Batch Processing with Spring Batch: Restartability, Chunking, Scaling, and Operations

Senior / Lead Java Developer Reading Chapter

Batch processing moves large finite datasets through a controlled workflow. Its design goals differ from online APIs: throughput, restartability, data quality, bounded resource use, audit, and completion deadlines matter more than millisecond response time. A strong batch job assumes interruption, duplicate input, bad records, partial progress, and operational reruns.

Job, JobInstance, JobExecution, and StepExecution

Spring Batch persists metadata describing job identity and execution.

```
Job:            invoiceGeneration
JobParameters:  businessDate=2026-08-10
JobInstance:    logical run for that identifying parameter set
JobExecution:   one attempt to execute the instance
StepExecution:  one attempt of a step inside that execution
```

Restart semantics depend on this identity. Launching with the same identifying parameters can restart a failed instance; changing them creates another logical instance. Do not add a random timestamp merely to bypass a completed job unless duplicate business execution is safe.

Metadata is operational state. Protect, back up, monitor, and clean it with an intentional retention policy.

Tasklet and Chunk-Oriented Steps

A tasklet performs one bounded unit such as moving a file or invoking a stored procedure. Chunk processing repeatedly reads items, optionally transforms them, and writes a group in one transaction.

```
read 100 items
-> process each
-> write 100
-> commit chunk and update restart metadata
-> repeat
```

Chunking balances commit overhead against rollback size, lock duration, memory, and restart granularity. A chunk of one gives fine recovery but high overhead; a huge chunk can roll back hours of work.

Reader, Processor, and Writer Contracts

`ItemReader` advances through input, `ItemProcessor` transforms or filters, and `ItemWriter` persists a chunk. Readers and writers often hold cursor, file, or paging state and participate in stream lifecycle.

Keep processors deterministic and side-effect free when possible. Fault-tolerant steps may reprocess items after rollback. A processor that sends email or mutates external state can duplicate effects.

Writers should use database constraints, idempotent keys, or conditional state transitions. The chunk transaction can roll back database work, but cannot recall a remote API accepted before failure.

Restartability

A restartable reader stores progress in the `ExecutionContext`, such as file line number, page key, or last processed identifier. State must be updated consistently with committed output.

```
chunk 1 commits -> checkpoint id=1000  
chunk 2 fails at id=1073  
restart -> resume after id=1000 and reprocess 1001 onward
```

Reprocessing a failed chunk is normal. The job must not assume every item executes exactly once. Stable item identity and idempotent output make restart safe.

When input can change between attempts, line number or offset may no longer identify the same record. Freeze input, version it, stage it into a durable table, or use a stable business key.

Job Parameters and Business Identity

Parameters should distinguish the business run: date, source file identity, tenant, or settlement cycle.

Separate identifying parameters from tuning values such as chunk size or worker count when the framework version supports that distinction.

Validate parameters before expensive work. Record input checksum, schema version, and source location. Two files with the same name but different content must not silently share a `JobInstance`.

Transaction Boundaries

Database readers and writers can participate in the chunk transaction. Cursor readers may hold database resources across chunks; paging readers issue deterministic page queries. The exact transaction and restart behavior depends on implementation.

Do not hold a database transaction while calling a slow remote service for every item. Stage remote requests, use an idempotent provider key, or separate the workflow into durable steps. If remote lookup is necessary, apply deadlines, rate limits, and cache only with defined freshness.

After a database exception, let the chunk roll back and create a fresh transaction for retry. Continuing in the same failed transaction or persistence context is unsafe.

Skip Versus Retry

Retry addresses transient failure. Skip accepts a known bad item and continues according to business policy.

```
deadlock / short outage -> retry with bounded backoff  
invalid postal code     -> skip or reject to quarantine  
programming defect      -> fail job
```

Classify exceptions precisely. Retrying deterministic validation wastes capacity; skipping database unavailability loses valid work. Configure limits because endless retry or skip can make a job appear active while producing no useful output.

Write skips are harder because one bad item can fail a chunk write. The framework may retry individual items to locate the failure, depending on configuration. Writers and processors must tolerate re-execution.

Quarantine and Data Quality

Skipped records need an owned destination with source identity, safe failure code, attempt information, and enough sanitized context for correction. Do not log full sensitive records or treat a skip count as sufficient evidence.

Define maximum skip rate and absolute count. A job that "succeeds" after skipping 40 percent of invoices is a business failure. Completion status should reflect policy and alert operators.

Correction and replay need stable item identity. A repaired record can be submitted to a dedicated recovery job rather than rerunning an entire historical file.

Idempotent Output

Use natural or generated business keys and database uniqueness:

```
insert into invoice(invoice_run_id, customer_id, ...)
values (:runId, :customerId, ...)
on conflict (invoice_run_id, customer_id) do nothing;
```

Alternatively use conditional update from an expected state, a processed-item table, or merge/upsert semantics whose overwrite rules are explicit.

Idempotency must include external effects. A stable run-item ID can become a payment or notification provider key. If the provider offers no idempotency, persist uncertain outcomes for reconciliation instead of blind replay.

Chunk Size Tuning

Chunk size affects:

- Commit and metadata overhead.
- Memory retained by items and persistence contexts.
- Database lock and log duration.
- Batch statement efficiency.
- Work repeated after failure.
- Time before progress becomes durable.

Measure realistic item size and writer behavior. One thousand tiny rows differs from one thousand documents containing megabytes. Tune alongside JDBC batch size and flush-clear interval.

JPA in Batch Jobs

A JPA persistence context retains managed entities and snapshots. Flush and clear per chunk or bounded interval to prevent heap growth. Avoid loading massive graphs and triggering N+1.

For simple transformations, JDBC or set-based SQL may be faster and easier to restart than entity-by-entity JPA. Choose entities when domain behavior and mappings justify them, not because the application already uses JPA.

Bulk JPQL or native updates bypass callbacks and managed state. Use a separate context and explicit audit or event strategy.

Paging and Keyset Reading

Offset paging becomes expensive at high offsets and unstable when rows change. A deterministic keyset reader advances through an indexed stable key:

```
select * from source
where id > :lastId
order by id
limit :pageSize;
```

Store `lastId` only after output commits. If input rows can be inserted behind the cursor or existing keys updated, define snapshot, high-water mark, status marker, or staging semantics.

Database cursor readers can stream efficiently but hold a connection and snapshot for a long time. That can pressure MVCC cleanup and recovery. Choose from input size, consistency requirement, and job duration.

File Input and Output

Files need identity, encoding, delimiter or schema version, header validation, checksum, and completion protocol. Do not consume a file while another system is still writing it. Use atomic rename, manifest, object-store completion marker, or immutable upload key.

Reject malformed structure before processing millions of rows. For output, write to a temporary name and publish atomically after successful completion. A partially written final file can be mistaken for complete downstream input.

Parallel Steps

Independent steps can run concurrently, for example customer and product reference loading. This shortens the critical path only if they do not contend for the same database, file, or downstream quota.

Define failure flow: if one parallel branch fails, which completed branch is rerun on restart? Completed steps are commonly skipped unless configured otherwise. Cleanup and compensation must be idempotent.

Multi-Threaded Steps

A multi-threaded step processes chunks concurrently within one process. Readers, processors, and writers must be thread-safe or scoped appropriately. Output order may differ from input order.

More threads can overwhelm the database or remote API. Bound concurrency to downstream capacity and coordinate database pool size. Measure lock contention and transaction-log pressure.

Shared mutable counters and non-thread-safe parsers cause data corruption. Prefer framework metrics and per-item immutable state.

Partitioning

Partitioning divides input into independent ranges, files, tenants, or shards managed by worker step executions.

```
manager step
-> partition A: customer IDs 1..1M
-> partition B: 1M..2M
-> partition C: 2M..3M
```

Partitions need non-overlap or idempotent overlap handling, balanced cost, durable identity, and independent restart. Equal ID ranges may be unbalanced when data is skewed. Partition from statistics or dynamic work queues where needed.

Remote partitioning sends partition metadata to workers; remote chunking keeps reading at the manager and sends chunks for processing. These approaches have different network, bottleneck, and restart behavior. Adopt them only after single-process bottlenecks are measured.

Scheduling and Single Ownership

A scheduler triggers work; Spring Batch tracks execution. Multiple scheduler replicas must not create duplicate JobInstances unintentionally. Use database-backed job identity and scheduler coordination rather than only an in-memory lock.

Misfires, daylight-saving transitions, and timezone changes matter. Schedule from an explicit zone and business calendar. A daily job should identify business date, not assume current wall-clock date at every worker.

Do not make application startup automatically launch every historical pending job unless that recovery policy is intentional.

Dependencies Between Jobs

Model dependencies through durable completion state, manifests, or orchestration. Sleeping and polling a filename without deadline or identity is fragile.

If Job B consumes Job A output, publish a completion fact only after A's output is durable and verified. Include version and checksum. B should reject incompatible or partial input.

A batch DAG needs cycle prevention, deadline propagation, and ownership when one upstream job misses its window.

Observability

Measure job and step duration, read/process/write counts, commit count, rollback, retry, skip, filter count, throughput, lag to deadline, chunk latency, and resource saturation.

Technical completion is not business completion. Report totals such as invoices generated, amount reconciled, unmatched records, and quarantine age. Use job and step names as metric labels, not unique execution IDs.

Logs should include execution identity and item key where safe. Traces are useful for remote calls but can be too expensive per item; sample or aggregate deliberately.

Operational Controls

Operators need safe commands to stop, abandon, restart, rerun as new instance, or replay quarantined items. These operations have different semantics and require authorization and audit.

Stopping should finish or roll back the current chunk, persist status, and release resources. Process termination without checkpoint increases replay. A start limit can prevent repeatedly running a dangerous step until manual correction.

Runbooks should state evidence to capture, restart parameters, expected duplicate protection, and how to verify outputs.

Testing Batch Jobs

Unit-test processors and validators directly. Step tests verify chunk, skip, retry, and transaction behavior. Job tests verify flow and restart metadata. Integration tests use real database constraints, representative files, and compatible brokers or providers.

Inject failures before and after chunk commit, during writer batch execution, after external acceptance, and during partition completion. Restart and prove final output has no loss or duplicate business effect.

Test volume, skew, poison records, disk full, database slowdown, scheduler duplicate trigger, and worker termination. A happy-path job over ten rows is not production evidence.

Test the interrupted execution

The most valuable restart test deliberately stops a job after several committed chunks. The test then launches the same business instance and verifies that completed work is not duplicated, failed work resumes from durable state, and totals reconcile with the input. A second test should fail after the external side effect but before the framework records success; this exposes whether idempotency is real or merely assumed.

Test data must include duplicate business keys, malformed records, transient database failures, permanently invalid records, and boundary values around page or partition splits. Assert both framework status and business outcome. A **COMPLETED** job with missing output is still a failed business process.

Metadata Repository and Retention

Spring Batch persists job and step metadata so that operators and the framework can reason about executions. That repository is operational data, not an unlimited audit archive. Frequent jobs can generate large execution-context and step-execution tables, slowing launches and operator queries. Define retention and archival based on investigation and compliance needs, and purge only executions that are no longer needed for restart.

The repository also participates in job-instance uniqueness and launch coordination. It must use a durable database with correct transaction isolation for the deployment. Multiple schedulers launching the same parameters should not create two independent business runs; uniqueness and ownership must be enforced rather than inferred from timing.

Remote Calls Inside a Batch Step

Calling an HTTP service from an item processor or writer couples chunk completion to network behavior. A large chunk may hold a database transaction while hundreds of remote calls execute, increasing lock time and making rollback expensive. A remote call may succeed even when the local transaction later fails, so repeating the chunk can repeat the side effect.

Prefer separating durable selection from remote delivery. Persist an outbox or work item in a short transaction, then let a bounded dispatcher call the remote system using an idempotency key. Record the provider reference and reconcile ambiguous timeouts. If direct calls are unavoidable, keep concurrency and timeouts bounded, use provider-supported idempotency, and design the writer for replay.

```
batch selection -> durable work/outbox -> commit
                  -> bounded dispatcher -> remote API
                  -> result/reconciliation record
```

Backpressure and Resource Governance

Batch systems run beside interactive services and can quietly consume all shared database connections, I/O bandwidth, or downstream quotas. Configure a separate pool or explicit concurrency budget so the batch cannot starve request traffic. Rate limiting should follow the downstream contract, while adaptive reduction can respond to latency and throttling.

More partitions are useful only while the slow resource has spare capacity. When database I/O is saturated, adding workers increases contention and restart complexity without increasing completed items per second. Measure item throughput, chunk duration, pool wait, lock time, and downstream throttling together.

Deployment Compatibility During Long Jobs

A long-running job can overlap a rolling application deployment. New code may read execution context written by an old version, and restarted chunks may use changed business logic. Store simple, versioned restart state rather than serialized implementation objects. Decide whether an in-progress execution must finish on the old version, can safely resume on the new version, or should be stopped and relaunched as a new business instance.

Database migrations also need backward compatibility for the duration of overlap. Destructive schema changes should wait until old workers and restartable executions no longer depend on the previous structure.

Common Senior-Level Traps

Traps include random job parameters to force reruns, processors with side effects, huge chunks, retrying validation errors, skipping infrastructure failures, cursor transactions lasting hours, and parallelism beyond database capacity.

Other traps include no quarantine owner, changing input between restart attempts, JPA contexts never cleared, output published before completion, and treating COMPLETED status as proof of business reconciliation.

A Strong Interview Answer

I identify a batch run by stable business parameters and design every step for interruption and restart. Chunk boundaries balance commit overhead, repeated work, locks, and memory. Readers checkpoint stable progress only with committed output; processors are deterministic; writers use idempotent keys and constraints. Retry is for transient failures, skip is an explicit data-quality policy with quarantine and limits. I scale through bounded threading or partitioning only after measuring downstream capacity and skew. JPA contexts are flushed and cleared, files use completion protocols, and external effects use stable idempotency. Operations expose progress, restart, stop, quarantine replay, and business reconciliation, all tested at crash boundaries.

Interview-Ready Summary

Reliable batch design assumes rerun and partial progress. Spring Batch metadata, chunk transactions, execution context, skip and retry provide mechanisms; stable identity and idempotent effects provide correctness.

Scale only after single-worker semantics are restartable. Completion must include business totals and reconciliation, not merely a green framework status.

Revision Notes

- JobInstance identity comes from the Job and identifying parameters.
- JobExecution represents one attempt of a logical instance.
- Chunk processing reads, processes, writes, and commits bounded groups.
- Chunk size controls overhead, rollback work, locks, and memory.
- Processors should be deterministic and side-effect free where possible.
- Restart checkpoints must align with committed output.
- Mutable input needs staging, versioning, or stable keys.
- Retry transient failures; skip only explicit bad-data categories.
- Fault-tolerant processing can re-execute items.
- Quarantine needs ownership, limits, repair, and replay.
- Writers require idempotent business keys and database constraints.
- Remote side effects need provider idempotency or reconciliation.
- Flush and clear JPA state during bounded chunks.
- Keyset readers need deterministic indexed progress.
- Long cursor transactions can retain snapshots and connections.
- File completion must be distinguishable from partial output.
- Parallelism must respect database and downstream capacity.
- Partitions need balanced, non-overlapping, restartable identity.
- Scheduling uses explicit business date and timezone.
- Measure business reconciliation as well as framework completion.